

AI Assisted Coding

Lab Assignment - 19.1

Name : Likhith Sai

Hall.t.No. : 2303A51026

Batch No. : 01

Lab 19 – Code Translation: Converting Between Programming Languages

Lab Objectives:

- Understand how AI tools can assist in translating code between different programming languages.
- Learn to verify correctness and functionality after translation.
- Explore syntactic and semantic differences between languages (e.g., Python, Java, C++).
- Practice debugging and optimizing AI-translated code.

Task 1 – Python to Java Conversion

Task: Translate a simple Python class into Java using AI assistance.

Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

Python code :

```
task_1.py > ...
1  # Translate a simple Python class into Java using AI assistance.
2
3  # Python Class
4  class Person:
5      def __init__(self, name, age):
6          self.name = name
7          self.age = age
8
9      def greet(self):
10         return f"Hello, my name is {self.name} and I am {self.age} years old."
11 # Example usage
12 person = Person("Alice", 30)
13 print(person.greet())
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2
/AI_Assisted_Coding/lab_assignment-19.1/task_1.py
Hello, my name is Alice and I am 30 years old.
```

Converted Java Code :

```
task_1.java
1  // Java Class (Translated from Python)
2  public class Person {
3      private String name;
4      private int age;
5
6      public Person(String name, int age) {
7          this.name = name;
8          this.age = age;
9      }
10
11     public String greet() {
12         return "Hello, my name is " + name + " and I am " + age + " years old.";
13     }
14
15     public static void main(String[] args) {
16         Person person = new Person("Alice", 30);
17         System.out.println(person.greet());
18     }
19 }
```

Output :

Output Generated files

```
Hello, my name is Alice and I am 30 years old.
```

Compiled and executed in 1.744 sec(s)

Task 2 – C++ to Python Function Conversion

Task: Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

C++ Code :

```
task_2.c++
1  // Convert a C++ function for factorial calculation into Python.
2
3  // C++ Code for Factorial Calculation
4  #include <iostream>
5  using namespace std;
6  int factorial(int n) {
7      if (n == 0) {
8          return 1;
9      }
10     return n * factorial(n - 1);
11 }
12 int main() {
13     int number;
14     cout << "Enter a number: ";
15     cin >> number;
16     cout << "Factorial of " << number << " is " << factorial(number) << endl;
17     return 0;
18 }
```

Output :

```
Output
Enter a number: 5
Factorial of 5 is 120

=== Code Execution Successful ===
```

Converted Python Code :

```
task_2.py > ...
1  # Python Code for Factorial Calculation
2  def factorial(n):
3      if n == 0:
4          return 1
5      return n * factorial(n - 1)
6  # Testing the function with sample inputs
7  if __name__ == "__main__":
8      number = int(input("Enter a number: "))
9      print(f"Factorial of {number} is {factorial(number)}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2
/AI_Assisted_Coding/lab_assignment-19.1/task_2.py
Enter a number: 5
Factorial of 5 is 120
```

Task 3 – SQL to Pandas Query

Task: Translate a SQL query into a Pandas DataFrame operation in Python.

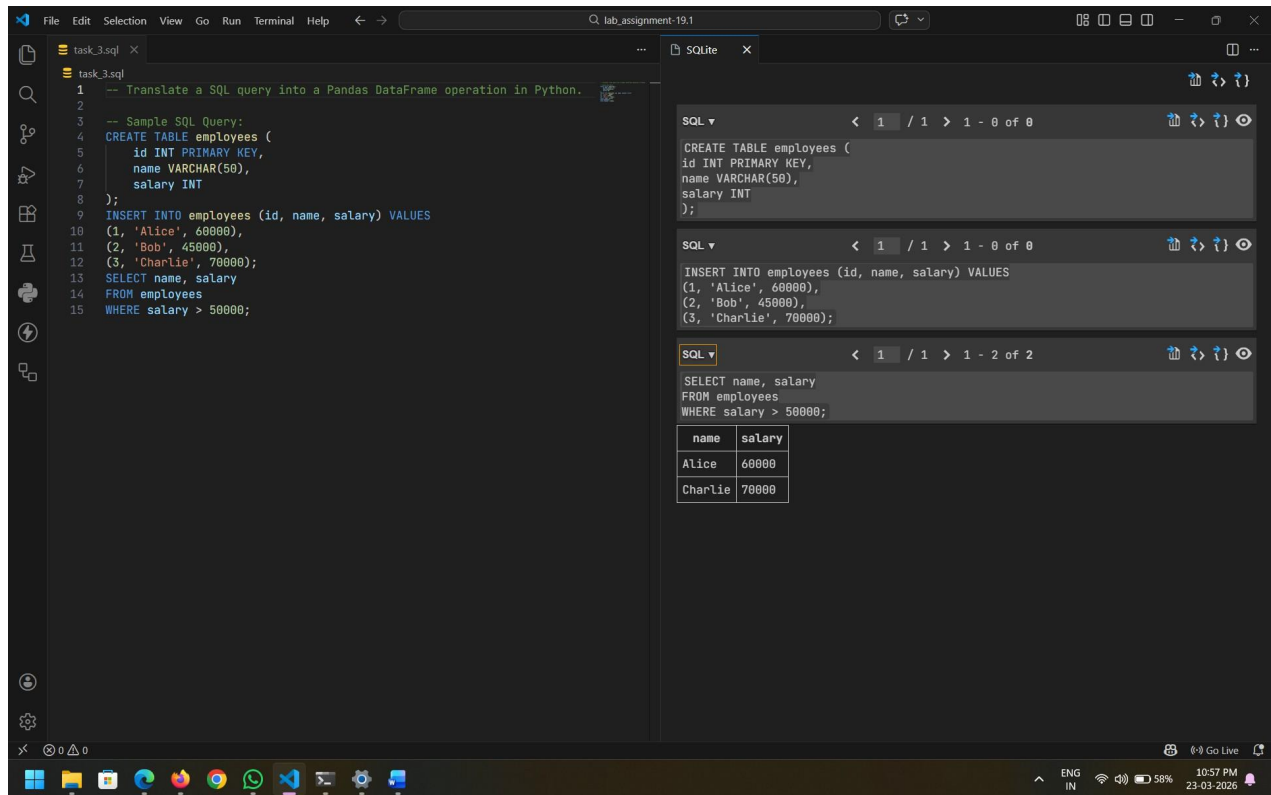
Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

SQL Code & Output :



The screenshot shows a VS Code editor with a file named `task_3.sql`. The code contains a comment, a sample SQL query to create a table, insert data, and filter results. The SQLite console on the right shows the execution of these queries and the resulting output table.

```
1 -- Translate a SQL query into a Pandas DataFrame operation in Python.
2
3 -- Sample SQL Query:
4 CREATE TABLE employees (
5     id INT PRIMARY KEY,
6     name VARCHAR(50),
7     salary INT
8 );
9 INSERT INTO employees (id, name, salary) VALUES
10 (1, 'Alice', 60000),
11 (2, 'Bob', 45000),
12 (3, 'Charlie', 70000);
13 SELECT name, salary
14 FROM employees
15 WHERE salary > 50000;
```

SQLite console output:

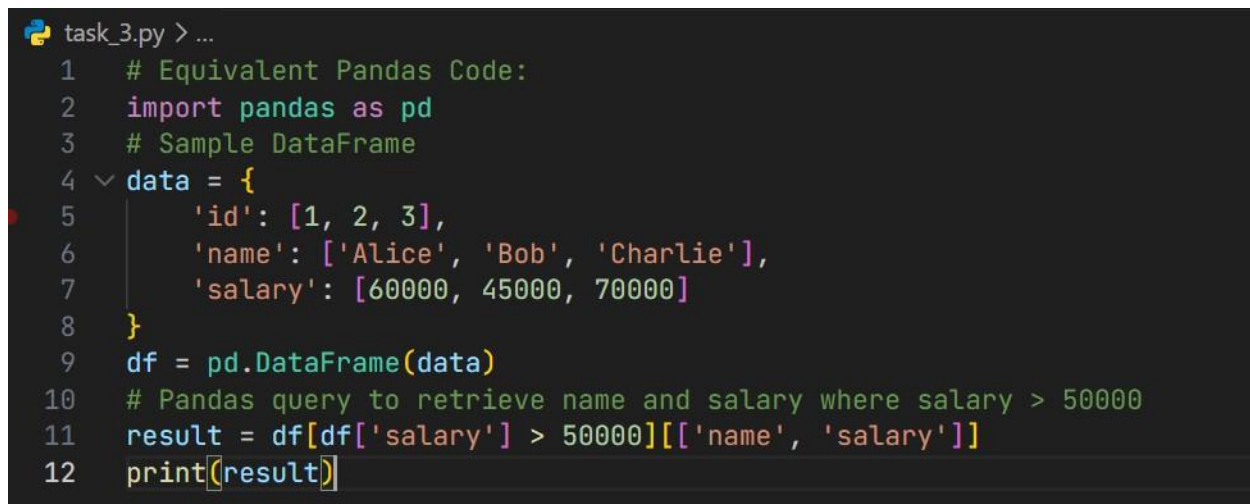
```
SQL > 1 / 1 > 1 - 0 of 0
CREATE TABLE employees (
id INT PRIMARY KEY,
name VARCHAR(50),
salary INT
);

SQL > 1 / 1 > 1 - 0 of 0
INSERT INTO employees (id, name, salary) VALUES
(1, 'Alice', 60000),
(2, 'Bob', 45000),
(3, 'Charlie', 70000);

SQL > 1 / 1 > 1 - 2 of 2
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

name	salary
Alice	60000
Charlie	70000

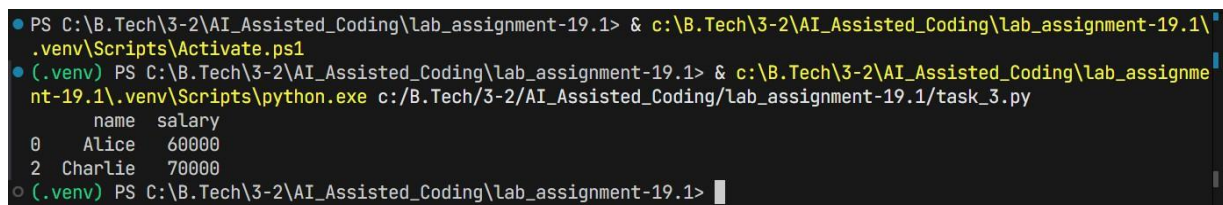
Converted Python Code :



The screenshot shows a Python script `task_3.py` that replicates the SQL logic using Pandas. It imports pandas, creates a DataFrame from a dictionary, and filters it to get names and salaries for those earning more than 50,000.

```
task_3.py > ...
1 # Equivalent Pandas Code:
2 import pandas as pd
3 # Sample DataFrame
4 data = {
5     'id': [1, 2, 3],
6     'name': ['Alice', 'Bob', 'Charlie'],
7     'salary': [60000, 45000, 70000]
8 }
9 df = pd.DataFrame(data)
10 # Pandas query to retrieve name and salary where salary > 50000
11 result = df[df['salary'] > 50000][['name', 'salary']]
12 print(result)
```

Output :



The screenshot shows a PowerShell terminal where the Python script is executed. The output displays the filtered data as a table with two rows: Alice with a salary of 60000 and Charlie with a salary of 70000.

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\
.venv\Scripts\Activate.ps1
(.venv) PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignme
nt-19.1\venv\Scripts\python.exe c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-19.1/task_3.py
   name  salary
0  Alice  60000
2  Charlie 70000
(.venv) PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1>
```

Task 4 – Real-Time Application: Multi-Language Snippet Converter

Scenario: Students will choose a real-time use case where they need code in multiple languages.

Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

Python web scraping snippet :

```
task_4.py > scrape_python
1  # Students will choose a real-time use case where they need code in multiple languages.
2  # Example:
3  # • Converting a Python web scraping snippet into JavaScript for browser automation.
4  # • Translating a Java method into C# for enterprise applications.
5
6  from numpy import require
7  import requests
8  from bs4 import BeautifulSoup
9  def scrape_python(url):
10     response = requests.get(url)
11     soup = BeautifulSoup(response.text, 'html.parser')
12     return soup.title.string
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\venv\Scripts\Activate.ps1
(.venv) PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1> & c:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-19.1\venv\Scripts\python.exe c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-19.1/task_4.py
```

Converted Javascript Code :

```
task_4.js > ...
1  // # JavaScript equivalent using Puppeteer for browser automation
2  // # Note: This requires Node.js and Puppeteer installed (npm install puppeteer)
3  const puppeteer = require('puppeteer');
4
5  async function scrape_javascript(url) {
6      const browser = await puppeteer.launch();
7      const page = await browser.newPage();
8      await page.goto(url);
9      const title = await page.title();
10     await browser.close();
11     return title;
12 }
13
14 // Example usage (in Node.js):
15 // scrape_javascript('https://example.com').then(title => console.log(title));
16
17 // # Challenges faced during translation:
18 // # 1. Syntax Differences: Python and JavaScript have different syntax and conventions, which required
19 // #   careful attention to detail during translation.
20 // # 2. Library Differences: The libraries used for web scraping in Python (BeautifulSoup) and JavaScript (Puppeteer)
21 // #   have different APIs and functionalities, which required understanding both libraries to ensure equivalent functionality.
22 // # 3. Asynchronous Programming: JavaScript's asynchronous nature with Promises and async/await required a different approach
23 // #   compared to Python's synchronous code, which added complexity to the translation process.
```